

Module M214

Application Cloud Native

E. Déployer sur Azure Cloud

Azure App Service — Applications multi-conteneurs

E.1

Introduire Azure Cloud
Concepts & Services

Docker Hub

Push d'une image
vers le registre

E.2

Azure App Service
Déploiement multi-ctn

OFPPT

Développement Digital — Option Full Stack

Contents

1	E.1 — Introduire Azure Cloud	2
1.1	Définition d’Azure Cloud	2
1.2	Composantes Principales d’Azure	2
1.2.1	Géographies, Régions et Zones de disponibilité	2
1.2.2	Groupes de Ressources	3
1.2.3	Azure Resource Manager (ARM)	3
1.2.4	Abonnements (Subscriptions)	4
1.3	Exemples de Services Offerts par Azure	5
1.4	Création d’un Compte Azure	5
2	Pousser une Image sur Docker Hub	7
2.1	Qu’est-ce que Docker Hub ?	7
2.2	Flux de déploiement	7
2.3	Création d’un compte Docker Hub	7
2.4	Taguer et pousser une image	7
3	E.2 — Azure App Service	9
3.1	Définition d’Azure App Service	9
3.2	Plans de Service (App Service Plan)	9
3.3	Déploiement d’une application mono-conteneur	9
3.4	Déploiement multi-conteneurs avec Docker Compose	11
3.5	Fichier docker-compose.yml pour Azure App Service	11
3.6	Gestion de l’application via le Portail Azure	13
3.7	Mettre à jour l’image déployée	13
3.8	Nettoyage des ressources Azure	14
4	Synthèse	14

1 E.1 — Introduire Azure Cloud

1.1 Définition d’Azure Cloud

Microsoft Azure

Microsoft Azure est la plateforme de **cloud computing** de Microsoft, lancée en 2010. Elle propose plus de **200 services** accessibles via Internet : hébergement d’applications, bases de données, intelligence artificielle, mise en réseau, sécurité, etc.

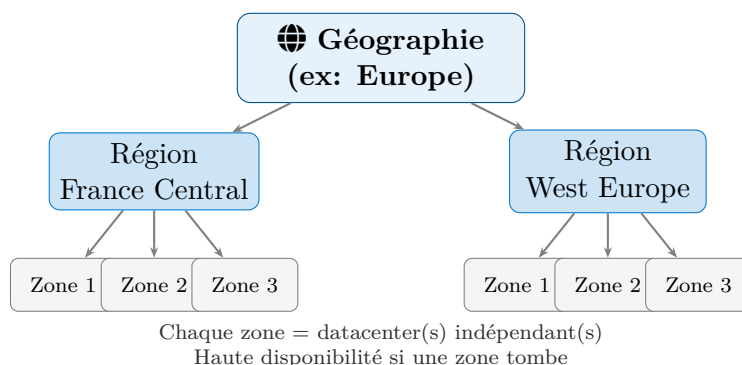
Cloud computing signifie qu’au lieu de gérer des serveurs physiques, vous louez des ressources informatiques à la demande, et vous ne payez que ce que vous consommez (*pay-as-you-go*).

Les trois modèles de cloud

Modèle	Description	Exemple Azure
IaaS	Infrastructure as a Service — vous gérez l’OS et les apps	Azure Virtual Machines
PaaS	Platform as a Service — vous gérez uniquement le code	Azure App Service
SaaS	Software as a Service — tout est géré par le fournisseur	Microsoft 365

1.2 Composantes Principales d’Azure

1.2.1 Géographies, Régions et Zones de disponibilité



Concept	Définition
Géographie	Zone continentale (Europe, Amériques, Asie...) qui regroupe des régions
Région	Emplacement physique regroupant plusieurs datacenters (ex : <i>France Central</i>)
Zone de dispo.	Datacenters séparés au sein d'une même région, pour la résilience

1.2.2 Groupes de Ressources

Groupe de Ressources (Resource Group)

Un **groupe de ressources** est un **conteneur logique** qui regroupe toutes les ressources Azure d'une application (App Service, base de données, stockage...).

Avantage : on peut **supprimer** tout le groupe d'un coup, voir les coûts groupés, et appliquer des permissions à l'ensemble.

Listing 1: Créer un groupe de ressources via Azure CLI

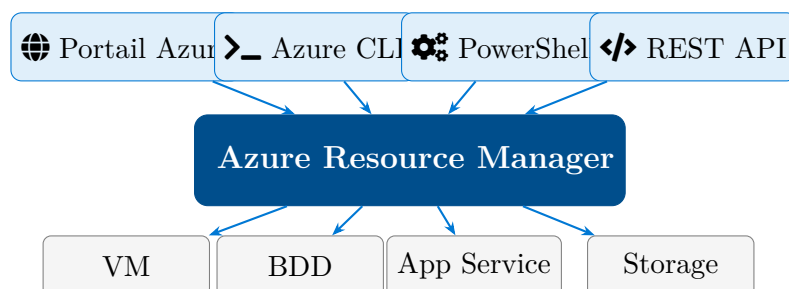
```
# Créer un groupe de ressources
az group create \
  --name mon-groupe \
  --location francecentral

# Lister les groupes existants
az group list --output table
```

1.2.3 Azure Resource Manager (ARM)

Azure Resource Manager

Azure Resource Manager (ARM) est la couche de gestion unifiée d'Azure. Toutes les actions (créer, modifier, supprimer une ressource) passent par ARM, que vous utilisiez le Portail web, l'Azure CLI, PowerShell ou une API REST.



1.2.4 Abonnements (Subscriptions)

Abonnement Azure

Un **abonnement** est un accord de facturation avec Microsoft Azure. Toutes les ressources créées sont rattachées à un abonnement, qui définit les limites de coût, les quotas et les permissions.

Un compte Azure peut avoir plusieurs abonnements (dev, test, production...).

Niveau	Description	Usage
Compte Azure	Identité principale	Un par personne / entreprise
Abonnement	Unité de facturation	Dev, Prod, Test...
Groupe de ressources	Conteneur logique	Par application ou projet
Ressource	Service créé	VM, App Service, BDD...

1.3 Exemples de Services Offerts par Azure

Catégorie	Services	Équivalent
Hébergement	Azure App Service, Azure Container Apps, AKS	Heroku, ECS
Base de données	Azure Cosmos DB, Azure SQL, Azure Database for PostgreSQL	MongoDB Atlas
Stockage	Azure Blob Storage, Azure Files	AWS S3
Registre	Azure Container Registry (ACR)	Docker Hub
Réseau	Azure Virtual Network, Load Balancer, DNS	AWS VPC
DevOps	Azure DevOps, GitHub Actions	Jenkins, GitLab CI
IA / ML	Azure OpenAI, Azure Cognitive Services	OpenAI API
Sécurité	Azure Key Vault, Azure Active Directory	AWS IAM

1.4 Création d'un Compte Azure

✓ Compte gratuit Azure — 200 \$ de crédits

Microsoft offre un **compte gratuit** pour les nouveaux utilisateurs :

- **200 \$ de crédits** utilisables pendant 30 jours
- Plus de **55 services gratuits** pendant 12 mois
- Certains services **toujours gratuits** (sans limite de durée)
- Aucun coût surpris : une carte bancaire est nécessaire pour l'inscription, mais elle ne sera pas débitée sans confirmation.

URL : <https://azure.microsoft.com/fr-fr/free>

1. Accédez à <https://azure.microsoft.com/fr-fr/free>
2. Cliquez sur “**Démarrer gratuitement**”
3. Connectez-vous avec un compte Microsoft (ou créez-en un)
4. Renseignez vos informations et une carte bancaire (vérification uniquement)
5. Validez votre téléphone par SMS
6. Accédez au **Portail Azure** : <https://portal.azure.com>

Listing 2: Installer et configurer l’Azure CLI

```
# Installation (Linux/macOS)
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# Windows : t\’el\’echarger depuis https://aka.ms/
installazurecliwindows
```

```
# Se connecter (ouvre le navigateur)  
az login  
  
# Vérifier l'abonnement actif  
az account show  
  
# Lister tous les abonnements  
az account list --output table
```

2 Pousser une Image sur Docker Hub

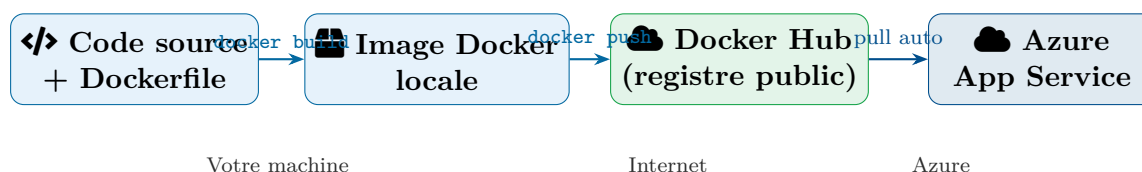
2.1 Qu'est-ce que Docker Hub ?

Docker Hub

Docker Hub (<https://hub.docker.com>) est le registre d'images Docker public et officiel. Il permet de **stocker**, **partager** et **télécharger** des images Docker.

Azure App Service peut déployer directement une image stockée sur Docker Hub, ce qui en fait l'intermédiaire idéal entre votre machine locale et Azure.

2.2 Flux de déploiement



2.3 Création d'un compte Docker Hub

1. Accédez à <https://hub.docker.com> et créez un compte gratuit.
2. Notez votre **nom d'utilisateur** Docker Hub (ex : monuser).
3. Créez un **dépôt (repository)** public pour chaque image.

2.4 Taguer et pousser une image

Listing 3: Pousser une image sur Docker Hub

```

# 1. Se connecter à Docker Hub
docker login
# Entrez votre username et password Docker Hub

# 2. Taguer l'image locale avec votre username Docker Hub
# Format : docker tag IMAGE_LOCALE USERNAME/NOM_IMAGE:TAG
docker tag mon-api:1.0 monuser/mon-api:1.0

# 3. Pousser l'image vers Docker Hub
docker push monuser/mon-api:1.0

# 4. Vérifier que l'image est visible sur hub.docker.com
# Accédez à : https://hub.docker.com/r/monuser/mon-api
  
```

⚠ Format du tag

Le tag doit suivre le format **USERNAME/NOM-IMAGE:VERSION**. **USERNAME** est votre identifiant Docker Hub (tout en minuscules). Sans ce format, **docker push** échouera.

Listing 4: Exemple complet build + tag + push

```
# Depuis le dossier de l'API
docker build -t monuser/tp-api:latest ./api

# Pousser directement
docker push monuser/tp-api:latest

# Vérifier l'image distante
docker pull monuser/tp-api:latest
```

3 E.2 — Azure App Service

3.1 Définition d’Azure App Service

Azure App Service

Azure App Service est un service **PaaS** (Platform as a Service) qui permet de déployer et d’héberger des applications web, des API REST et des applications mobiles **sans gérer l’infrastructure sous-jacente**.

Il supporte de nombreux langages et runtimes : Node.js, Python, PHP, Java, .NET, Ruby — et surtout des **conteneurs Docker**.

3.2 Plans de Service (App Service Plan)

App Service Plan

Un **App Service Plan** définit les ressources physiques (CPU, RAM, stockage) allouées à votre application. Plusieurs applications peuvent partager le même plan.

Niveau	CPU / RAM	Caractéristiques	Usage
F1 (Free)	60 min/jour	Partagé, pas de domaine custom	Tests
B1 (Basic)	1 vCPU, 1.75 Go	Dédié, domaine custom	Dév / TP
S1 (Standard)	1 vCPU, 1.75 Go	Auto-scale, slots de déploiement	Production
P1v3 (Premium)	2 vCPU, 8 Go	Haute performance	Production intensive

⚠ Plan gratuit et conteneurs

Le plan **F1 (Free)** ne supporte pas les conteneurs Docker. Pour déployer une application conteneurisée, utilisez au minimum le plan **B1 (Basic)**, qui est éligible aux crédits Azure gratuits.

3.3 Déploiement d’une application mono-conteneur

Listing 5: Déployer un conteneur Docker via Azure CLI

```
# 1. Créer un groupe de ressources
az group create \
  --name rg-tp-api \
  --location francecentral

# 2. Créer un App Service Plan (niveau B1, Linux)
az appservice plan create \
  --name plan-tp-api \
```

```
--resource-group rg-tp-api \  
--sku B1 \  
--is-linux  
  
# 3. Créer l'App Service avec l'image Docker Hub  
az webapp create \  
  --name mon-api-app \  
  --resource-group rg-tp-api \  
  --plan plan-tp-api \  
  --deployment-container-image-name monuser/mon-api:latest  
  
# 4. Ouvrir l'application dans le navigateur  
az webapp browse \  
  --name mon-api-app \  
  --resource-group rg-tp-api
```

3.4 Déploiement multi-conteneurs avec Docker Compose

Multi-conteneurs sur App Service

Azure App Service supporte le déploiement d'applications **multi-conteneurs** via un fichier `docker-compose.yml`. Tous les services déclarés tournent dans le même environnement App Service.

⚠ Limitations multi-conteneurs sur App Service

- Seul le pilote réseau **bridge** est supporté.
- Les **volumes** sont limités aux bind mounts (pas de volumes nommés pers.).
- Les **ports exposés** ne sont accessibles que sur le port **80** ou **8080**.
- Une seule image peut être issue d'un **build**: local — les autres doivent être sur un registre (Docker Hub ou ACR).
- **RabbitMQ** nécessite un plan de service élevé — en TP, utiliser une image MongoDB Atlas ou Cosmos DB à la place.

3.5 Fichier docker-compose.yml pour Azure App Service

```

1 version: '3.8'
2
3 services:
4
5   # API Node.js - image depuis Docker Hub
6   api:
7     image: monuser/tp-api:latest      # Image publique sur Docker
      Hub
8     ports:
9       - "80:3000"                    # Azure attend le port 80 ou
      8080
10    environment:
11      - NODE_ENV=production
12      - PORT=3000
13      - MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/
      produitsdb
14    restart: always
15
16   # MongoDB - image officielle
17   mongodb:
18     image: mongo:7.0
19     environment:
20       MONGO_INITDB_ROOT_USERNAME: admin
21       MONGO_INITDB_ROOT_PASSWORD: admin123
22     restart: always

```

Listing 6: docker-compose.yml adapté pour Azure App Service

Listing 7: Déployer le multi-conteneur via Azure CLI

```

# 1. Créer l'App Service avec configuration multi-conteneurs
az webapp create \
  --name mon-app-multicontainer \

```

```
--resource-group rg-tp-api \  
--plan plan-tp-api \  
--multicontainer-config-type compose \  
--multicontainer-config-file docker-compose.yml  
  
# 2. Définir les variables d'environnement  
az webapp config appsettings set \  
  --name mon-app-multicontainer \  
  --resource-group rg-tp-api \  
  --settings NODE_ENV=production PORT=3000  
  
# 3. Activer les logs de conteneur  
az webapp log config \  
  --name mon-app-multicontainer \  
  --resource-group rg-tp-api \  
  --docker-container-logging filesystem  
  
# 4. Consulter les logs en temps réel  
az webapp log tail \  
  --name mon-app-multicontainer \  
  --resource-group rg-tp-api
```

3.6 Gestion de l'application via le Portail Azure

Après déploiement, le Portail Azure (<https://portal.azure.com>) permet de :

Section du Portail	Ce que vous pouvez faire
Vue d'ensemble	URL de l'app, statut, redémarrer, arrêter
Paramètres d'application	Variables d'environnement (secrets, URLs BDD)
Centre de déploiement	Changer l'image, configurer CI/CD depuis GitHub
Flux de journaux	Voir les logs en temps réel
Mise à l'échelle	Augmenter le nombre d'instances (scale out)

3.7 Mettre à jour l'image déployée

Listing 8: Cycle de mise à jour complet

```
# 1. Modifier le code localement

# 2. Rebuild l'image avec un nouveau tag
docker build -t monuser/tp-api:2.0 ./api

# 3. Pousser la nouvelle version sur Docker Hub
docker push monuser/tp-api:2.0

# 4. Mettre à jour l'image sur Azure App Service
az webapp config container set \
  --name mon-app-multicontainer \
  --resource-group rg-tp-api \
  --docker-custom-image-name monuser/tp-api:2.0

# 5. Redémarrer l'application
az webapp restart \
  --name mon-app-multicontainer \
  --resource-group rg-tp-api
```

3.8 Nettoyage des ressources Azure

✖ Supprimer les ressources pour éviter les frais !

Sur Azure, les ressources en cours d'exécution consomment des crédits même si vous ne les utilisez pas. Supprimez toujours les ressources à la fin d'un TP ou d'un projet de test.

Listing 9: Supprimer un groupe de ressources complet

```
# Supprimer tout le groupe de ressources (App Service Plan + App
+ ...)
az group delete \
  --name rg-tp-api \
  --yes \
  --no-wait

# Vérifier que le groupe a bien été supprimé
az group list --output table
```

4 Synthèse

Points clés à retenir

1. **Azure** est une plateforme cloud de plus de 200 services (IaaS, PaaS, SaaS)
2. L'unité organisationnelle est : **Compte** → **Abonnement** → **Groupe de ressources** → **Ressource**
3. **Docker Hub** est le registre intermédiaire entre votre machine et Azure
4. Le flux de déploiement : `docker build` → `docker push` → Azure App Service pull l'image
5. **Azure App Service** est un PaaS qui héberge les apps sans gérer les serveurs
6. Le multi-conteneur utilise un fichier `docker-compose.yml` avec les images depuis Docker Hub
7. Toujours **supprimer les ressources** après un TP pour éviter la facturation

Code+ Docker → `docker build` → `image locale` → `docker push` → Docker Hub → Azure App Service

Cycle complet : Code local → Registre → Cloud